

CSES Sliding Window & Two Pointers

A Problem-Solving Companion to the CSES Problem Set

Maruri Saikiran

August 10, 2026

Contents

1	Sliding Window Problems	1
1.1	The Add / Remove / Report Template	1
1.2	Sliding Window Sum	2
1.3	Sliding Window Minimum	3
1.4	Sliding Window XOR	4
1.5	Sliding Window OR	6
1.6	Sliding Window Distinct Values	7
1.7	Sliding Window Mode	9
1.8	Sliding Window Mex	10
1.9	Sliding Window Median	12
1.10	Sliding Window Cost	14
1.11	Sliding Window Inversions	16
1.12	Sliding Window Advertisement (Maximum)	18
2	Two Pointers and Subarrays	21
2.1	Two Pointers vs Prefix Sums	21
2.2	Subarray Sums I	22
2.3	Subarray Sums II	23
2.4	Subarray Divisibility	24
2.5	Subarray Distinct Values	26
2.6	Playlist	27
2.7	Maximum Subarray Sum	29

Chapter 1

Sliding Window Problems

A sliding window turns a quadratic “look at every subarray” into a linear “maintain an answer as the window moves.” The whole craft is choosing a data structure that supports the three window operations cheaply: add the entering element, remove the leaving element, and report the current answer. Pick the right structure and each of these problems is the same loop with a different helper.

1.1 The Add / Remove / Report Template

Every fixed-size sliding-window problem here processes windows of width k by advancing one step at a time: the right end gains an element, the left end loses one, and we output the window’s answer. The only thing that varies is *which structure* maintains that answer under those two mutations.

Match the Answer to a Structure That Supports Removal

- **Sum / XOR** – invertible: add the new, subtract/xor out the old in $O(1)$.
- **Min / Max** – a **monotonic deque** gives $O(1)$ amortised.
- **OR / bit aggregates** – keep per-bit counts, since OR is not invertible.
- **Distinct count / mode / mex** – a hash map of frequencies, sometimes with “count of counts” buckets or a set of absent values.
- **Median / cost** – two balanced multisets split at the median, with running sums.
- **Inversions** – a Fenwick tree over value ranks, updated incrementally as elements enter and leave.

Why Removal Is the Hard Part

Adding an element to a window is usually easy; *removing* the element that falls off the left is what dictates the data structure. Sums and XORs are invertible so removal is trivial; min/max are not, which is why they need a monotonic deque; order statistics need a balanced multiset or a Fenwick tree. Reading a problem, ask first: “how do I *undo* the leftmost element?”

The sections cover, in order: window sum, minimum, XOR, OR, distinct values, mode, mex, median, cost, inversions, and the advertisement (maximum) variant. Each states the structure, the three window operations, a dry run, complete C++, and complexity.

1.2 Sliding Window Sum

Problem Statement. Given an array of n integers and a window size k , output the sum of every contiguous window of size k (there are $n - k + 1$ of them).

Pattern Recognition

Sum is **invertible**, so the window answer updates in $O(1)$: when the window slides one step, add the new right element and subtract the old left element. This is the simplest instance of the add/remove/report template.

A running total

Compute the sum of the first window directly. Then each slide changes the window by exactly two elements: one enters on the right, one leaves on the left. Because addition has an inverse, the new sum is $sum + a[\text{new}] - a[\text{old}]$ – no re-summation. (Equivalently, a prefix-sum array answers each window as $pre[i + k] - pre[i]$.)

Invertible Aggregates Slide for Free

Any aggregate with an inverse – sum, XOR, product over a field – lets the window update in constant time by undoing the departing element and applying the arriving one. Reach for this before any heavier structure.

Algorithm

1. Sum the first k elements.
2. For each subsequent window, add $a[i]$ and subtract $a[i - k]$; record the sum.

Dry run

Dry Run. $a = [1, 3, 2, 5, 1]$, $k = 3$.

- Window $[1, 3, 2]$: sum 6.
- Slide: $+5 - 1 \Rightarrow [3, 2, 5]$ sum 10.
- Slide: $+1 - 3 \Rightarrow [2, 5, 1]$ sum 8. Output 6, 10, 8.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<long long> a(n);
8     for (auto& x : a) cin >> x;
9
10    long long sum = 0;
11    for (int i = 0; i < k; i++) sum += a[i];
12    cout << sum << " ";
13    for (int i = k; i < n; i++) {
14        sum += a[i] - a[i - k];           // add new, drop old

```

```

15     cout << sum << " ";
16 }
17 cout << "\n";
18 return 0;
19 }

```

Complexity Analysis

Time Complexity. $O(n)$ – one pass with $O(1)$ per window. **Space Complexity.** $O(1)$ beyond the input.

Key Takeaway

When the window statistic is invertible, sliding is a constant-time add-and-subtract. Establish this baseline first; the harder window problems are exactly those where the statistic (min, mode, median) has no cheap inverse and demands a supporting structure.

1.3 Sliding Window Minimum

Problem Statement. Given an array of n integers and a window size k , output the minimum of every contiguous window of size k .

Pattern Recognition

Minimum has no cheap inverse, so a running value fails. The right tool is a **monotonic deque** that keeps candidate minima in increasing order: the front is always the current window's minimum, and each element is pushed and popped at most once, giving $O(n)$ overall.

The monotonic deque

Maintain a deque of *indices* whose values are strictly increasing from front to back. When a new element arrives, pop from the back every index whose value is $\geq$ the new value – those can never be the minimum again while the newer, smaller element is in the window. Then push the new index. Pop from the front any index that has slid out of the window ($\text{index} \leq i - k$). The front index's value is the window minimum.

Discard Dominated Candidates

An older element that is larger than a newer one is “dominated”: as long as the newer element is present, the older can never be the minimum, so it is removed from the back forever. Each index enters and leaves the deque once, which is why the total work is linear despite the inner popping.

Algorithm

1. For each index i : pop back while its value $\geq a[i]$; push i .
2. Pop front while it is $\leq i - k$ (out of window).
3. Once $i \geq k - 1$, output $a[\text{front}]$.

Dry run

Dry Run. $a = [2, 1, 3, 4, 1]$, $k = 2$.

- $i=0$: deque $[0]$. $i=1$: $a[1]=1 \leq a[0]$, pop 0, push 1; window min $a[1] = 1$.
- $i=2$: push 2 (deque $[1, 2]$, but 1 now out at $i - k = 0$? front 1 > 0 stays); min $a[1] = 1$. Then front $1 \leq 2 - 2 = 0$? no.
- Continuing yields minima 1, 1, 3, 1 for the four windows.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    deque<int> dq;                                // indices, values
11    // increasing
12    for (int i = 0; i < n; i++) {
13        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
14        dq.push_back(i);
15        if (dq.front() <= i - k) dq.pop_front(); // drop out-of-
16        // window
17        if (i >= k - 1) cout << a[dq.front()] << " ";
18    }
19    cout << "\n";
20    return 0;
21 }
```

Complexity Analysis

Time Complexity. $O(n)$ – each index is pushed and popped at most once. **Space Complexity.** $O(k)$ for the deque.

Key Takeaway

The monotonic deque is the canonical answer to sliding min/max: keep only non-dominated candidates in monotonic order, front is the answer. It is the single most reused sliding-window structure and appears again as the maximum (advertisement) variant.

1.4 Sliding Window XOR

Problem Statement. Given an array of n integers and a window size k , output the bitwise XOR of every contiguous window of size k .

Pattern Recognition

XOR is its own inverse ($x \oplus x = 0$), so, like sum, the window XOR slides in $O(1)$: XOR in the entering element and XOR out the leaving one. Equivalently, a **prefix-XOR** array answers each window as $pre[i + k] \oplus pre[i]$.

Prefix XOR

Let $pre[0] = 0$ and $pre[i] = a[0] \oplus \dots \oplus a[i-1]$. Then the XOR of $a[l..r]$ is $pre[r+1] \oplus pre[l]$, because the shared prefix $a[0..l-1]$ cancels itself (each bit XORed twice returns to 0). A window of size k starting at i is $pre[i+k] \oplus pre[i]$.

Self-Inverse Means Cancellation

Because $x \oplus x = 0$, overlapping prefixes cancel exactly, so a prefix-XOR array reduces any window XOR to a single operation. The same self-inverse property lets a running XOR drop the departing element by simply XORing it again.

Algorithm

1. Build the prefix-XOR array (or maintain a running XOR).
2. For each window i , output $pre[i+k] \oplus pre[i]$.

Dry run

Dry Run. $a = [1, 2, 3, 4]$, $k = 2$.

- $pre = [0, 1, 3, 0, 4]$.
- Window $[1, 2]$: $pre[2] \oplus pre[0] = 3$. $[2, 3]$: $pre[3] \oplus pre[1] = 0 \oplus 1 = 1$. $[3, 4]$: $pre[4] \oplus pre[2] = 4 \oplus 3 = 7$.
- Output 3, 1, 7.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    vector<int> pre(n + 1, 0);
11    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] ^ a[i];
12
13    for (int i = 0; i + k <= n; i++)
14        cout << (pre[i + k] ^ pre[i]) << " ";    // window XOR
15    cout << "\n";
16    return 0;
17 }
```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(n)$ for the prefix array (or $O(1)$ with a running XOR).

Key Takeaway

XOR joins sum in the “invertible, slides in $O(1)$ ” category thanks to being its own inverse. Prefix XOR is the range-query form and reappears in counting subarrays with a target XOR. Contrast with OR (next), which is *not* invertible and needs per-bit counts.

1.5 Sliding Window OR

Problem Statement. Given an array of n integers and a window size k , output the bitwise OR of every contiguous window of size k .

Pattern Recognition

OR is **not invertible**: removing an element cannot simply undo its bits, because another element in the window may still set them. The fix is to track, for each bit position, *how many* window elements set that bit – then the OR has a bit set exactly when its count is positive.

Per-bit counts

Maintain an array $cnt[b]$ = number of elements in the current window whose bit b is set. Adding an element increments $cnt[b]$ for each of its set bits; removing an element decrements them. The OR of the window has bit b set iff $cnt[b] > 0$. Reconstruct the OR by summing 2^b over all bits with positive count. This restores $O(1)$ amortised updates (constant number of bits) despite OR’s lack of an inverse.

Count Contributors per Bit

When an aggregate is not invertible, replace “does any element set this bit?” with “how many do?” A count is invertible (increment/decrement), and the OR is recovered by testing each count against zero. The same idea handles AND (count zeros) and other non-invertible bitwise folds.

Algorithm

1. For the first window, increment $cnt[b]$ for every set bit of each element.
2. To slide: add the new element’s bits, remove the old element’s bits.
3. Output $\sum_{b: cnt[b] > 0} 2^b$ for each window.

Dry run

Dry Run. $a = [1, 2, 3]$ (bits: 1=001, 2=010, 3=011), $k = 2$.

- Window $[1, 2]$: bit0 count 1, bit1 count 1 $\Rightarrow$ OR = 3.
- Slide to $[2, 3]$: remove 1 (bit0 $\rightarrow$ 0), add 3 (bit0 $\rightarrow$ 1, bit1 $\rightarrow$ 2) $\Rightarrow$ bits 0, 1 set, OR = 3.
- Output 3, 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    const int BITS = 31;
11    vector<int> cnt(BITS, 0);
12    auto add = [&](int x, int delta) {
13        for (int b = 0; b < BITS; b++) if (x & (1 << b)) cnt[b] +=
            delta;
14    };
15    auto currentOR = [&]() {
16        int res = 0;
17        for (int b = 0; b < BITS; b++) if (cnt[b] > 0) res |= (1 << b);
18        return res;
19    };
20
21    for (int i = 0; i < k; i++) add(a[i], +1);
22    cout << currentOR() << " ";
23    for (int i = k; i < n; i++) {
24        add(a[i], +1); add(a[i - k], -1);           // add new, remove
            old
25        cout << currentOR() << " ";
26    }
27    cout << "\n";
28    return 0;
29 }

```

Complexity Analysis

Time Complexity. $O(n \cdot \text{BITS})$ – constant 31 bits per element. **Space Complexity.** $O(\text{BITS})$.

Key Takeaway

Non-invertible aggregates like OR are made sliding-friendly by counting contributors per bit, so removal becomes a decrement. This “count-then-threshold” conversion is the general recipe whenever an operation lacks an inverse.

1.6 Sliding Window Distinct Values

Problem Statement. Given an array of n integers and a window size k , output the number of distinct values in every contiguous window of size k .

Pattern Recognition

Distinct-count slides cleanly with a **frequency hash map**: track how many times each value appears in the window. A value contributes to the distinct count exactly while its frequency is positive, so the count changes only when a frequency crosses between 0 and 1.

Frequency map with a distinct counter

Keep $freq[v]$ and a running $distinct$. Adding a value v : if its frequency was 0, it is newly present, so increment $distinct$; then increment $freq[v]$. Removing a value v : decrement $freq[v]$; if it drops to 0, the value has left the window, so decrement $distinct$. Each slide performs one add and one remove, both $O(1)$ amortised.

Watch the Zero Boundary

The distinct count only moves when a frequency transitions $0 \rightarrow 1$ (a value appears) or $1 \rightarrow 0$ (a value disappears). Checking that boundary on each add/remove keeps the counter exact without rescanning the window.

Algorithm

1. Add the first k elements, updating $freq$ and $distinct$.
2. For each slide: add $a[i]$ (increment distinct if it was absent), remove $a[i - k]$ (decrement distinct if it becomes absent).
3. Output $distinct$ per window.

Dry run

Dry Run. $a = [1, 2, 1, 3]$, $k = 3$.

- Window $[1, 2, 1]$: freqs $\{1:2, 2:1\}$, distinct = 2.
- Slide to $[2, 1, 3]$: remove a 1 (freq $2 \rightarrow 1$, still present), add 3 (new) $\Rightarrow$ distinct = 3.
- Output 2, 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    unordered_map<int, int> freq;
11    int distinct = 0;
12    auto add = [&](int v) { if (freq[v]++ == 0) distinct++; };
13    auto remove = [&](int v) { if (--freq[v] == 0) distinct--; };
14
15    for (int i = 0; i < k; i++) add(a[i]);
16    cout << distinct << " ";
17    for (int i = k; i < n; i++) {
18        add(a[i]); remove(a[i - k]);
19        cout << distinct << " ";
20    }
21    cout << "\n";
22    return 0;
23 }
```

Complexity Analysis

Time Complexity. $O(n)$ amortised with a hash map. **Space Complexity.** $O(k)$ distinct keys at most.

Key Takeaway

Distinct-value counting is a frequency map plus a counter that only reacts at the zero boundary. This add/remove-with-boundary-check idiom generalises to “count of values with property P” whenever P depends on a per-value frequency.

1.7 Sliding Window Mode

Problem Statement. Given an array of n integers and a window size k , output the mode of every contiguous window of size k – the value occurring most often (if several tie, the smallest such value).

Pattern Recognition

The mode must survive both additions and *removals*, so a single “max frequency” is not enough. Keep frequency buckets: for each frequency f , an ordered set of the values that currently occur f times. The mode is the smallest value in the highest non-empty bucket.

Frequency buckets of values

Maintain $freq[v]$ and $bucket[f]$ = an ordered set of values whose current frequency is exactly f , plus $maxFreq$. Adding a value v moves it from $bucket[freq[v]]$ to $bucket[freq[v] + 1]$ and may raise $maxFreq$. Removing v moves it down a bucket; if that empties the top bucket, $maxFreq$ decreases. The mode is the smallest value in $bucket[maxFreq]$, read in $O(\log)$ from the ordered set’s begin.

Buckets Make the Mode Removal-Safe

The trouble with modes under removal is that the top frequency can drop. Grouping values by their exact frequency in ordered buckets lets a removal adjust $maxFreq$ in $O(1)$ and still expose the smallest tied value instantly – something a lone counter cannot do.

Algorithm

1. Maintain $freq$, $bucket[f]$ (ordered set of values), and $maxFreq$.
2. On add/remove of v : erase v from its old bucket, change its frequency, insert into the new bucket, and update $maxFreq$.
3. The mode is $*bucket[maxFreq].begin()$; output it per window.

Dry run

Dry Run. $a = [1, 2, 2, 1, 1]$, $k = 3$.

- $[1, 2, 2]$: freqs 1:1, 2:2; $maxFreq$ 2, $bucket[2] = \{2\}$; mode 2.
- Slide to $[2, 2, 1]$: still 2:2, 1:1; mode 2.
- Slide to $[2, 1, 1]$: 1:2, 2:1; $bucket[2] = \{1\}$; mode 1. Output 2, 2, 1.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    unordered_map<int, int> freq;
11    map<int, set<int>>> bucket;           // frequency -> values
12    int maxFreq = 0;
13
14    auto change = [&](int v, int delta) {
15        int f = freq[v];
16        if (f > 0) bucket[f].erase(v);
17        if (f > 0 && bucket[f].empty()) bucket.erase(f);
18        f += delta;
19        freq[v] = f;
20        if (f > 0) bucket[f].insert(v);
21        maxFreq = bucket.empty() ? 0 : bucket.rbegin()->first;
22    };
23
24    for (int i = 0; i < k; i++) change(a[i], +1);
25    cout << *bucket[maxFreq].begin() << " ";
26    for (int i = k; i < n; i++) {
27        change(a[i], +1); change(a[i - k], -1);
28        cout << *bucket[maxFreq].begin() << " ";
29    }
30    cout << "\n";
31    return 0;
32 }

```

Complexity Analysis

Time Complexity. $O(n \log k)$ – each add/remove touches ordered sets. **Space Complexity.** $O(k)$.

Key Takeaway

Tracking the mode under removals needs frequency-to-values buckets so the top frequency and its smallest value are both maintainable. The “count of counts” idea – indexing by frequency, not by value – is the general trick for order statistics on frequencies.

1.8 Sliding Window Mex

Problem Statement. Given an array of n non-negative integers and a window size k , output the mex of every contiguous window – the smallest non-negative integer *not* present in the window.

Pattern Recognition

The mex of a size- k window is at most k (with k elements, some value in $0 \dots k$ must be missing). So track only values in $[0, k]$ with a frequency map and keep an ordered **set of**

absent values; the mex is its smallest element.

A set of the missing small values

Values larger than k can never be the mex, so ignore them. Initialise a set $absent = \{0, 1, \dots, k\}$ and a frequency map. When a value $v \leq k$ enters and its frequency goes $0 \rightarrow 1$, erase v from $absent$ (now present). When it leaves and its frequency returns to 0, insert v back into $absent$. The mex is the smallest element of $absent$ – read in $O(\log k)$ from its begin.

Bound the Universe, Then Track Absence

Because the mex cannot exceed the window size, the relevant universe is just $[0, k]$. Maintaining which of those are absent – rather than which are present – puts the answer (the smallest absent value) at the front of an ordered set.

Algorithm

1. $absent = \{0, \dots, k\}$; empty frequency map.
2. Add the first k elements (erasing from $absent$ on $0 \rightarrow 1$).
3. Each slide: add $a[i]$, remove $a[i - k]$ (inserting into $absent$ on $1 \rightarrow 0$); output $*absent.begin()$.

Dry run

Dry Run. $a = [0, 1, 3, 0]$, $k = 3$.

- $absent = \{0, 1, 2, 3\}$. Window $[0, 1, 3]$: erase 0, 1, 3; $absent = \{2\}$; mex 2.
- Slide to $[1, 3, 0]$: remove first 0 (freq 0, reinsert 0), add 0 (erase 0 again); $absent = \{2\}$; mex 2.
- Output 2, 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    set<int> absent;
11    for (int v = 0; v <= k; v++) absent.insert(v);
12    unordered_map<int, int> freq;
13
14    auto add = [&](int v) {
15        if (v <= k && freq[v]++ == 0) absent.erase(v);
16    };
17    auto remove = [&](int v) {
18        if (v <= k && --freq[v] == 0) absent.insert(v);
19    };

```

```

20
21     for (int i = 0; i < k; i++) add(a[i]);
22     cout << *absent.begin() << " ";
23     for (int i = k; i < n; i++) {
24         add(a[i]); remove(a[i - k]);
25         cout << *absent.begin() << " ";
26     }
27     cout << "\n";
28     return 0;
29 }

```

Complexity Analysis

Time Complexity. $O(n \log k)$ – ordered-set operations bounded by the $[0, k]$ universe.

Space Complexity. $O(k)$.

Key Takeaway

Mex problems shrink to a bounded universe (here $[0, k]$) and then track *absence* in an ordered set so the answer is the set's minimum. Bounding the universe before choosing a structure is the recurring first move for mex.

1.9 Sliding Window Median

Problem Statement. Given an array of n integers and a window size k , output the median of every contiguous window (for even k , the lower of the two middle values).

Pattern Recognition

The median needs the middle element under both insertion and deletion. Two balanced **multisets** – a *low* half and a *high* half – split at the median do exactly this: keep the low half's size equal to (or one more than) the high half, and the median is the low half's maximum.

Two balanced halves

Maintain *low* (a multiset of the smaller elements) and *high* (the larger elements) with the invariant $|low| = \lceil m/2 \rceil$ for a window of size m . Insertion routes the new value to the correct half and then rebalances by moving one element across if sizes drift. Deletion (when the left element leaves) erases it from whichever half holds it – a multiset supports erase-by-value in $O(\log k)$ – and rebalances. The median is always $*low.rbegin()$ (its largest element).

Balanced Multisets Give Erasable Heaps

Ordinary heaps cannot delete an arbitrary element, which sliding windows require. Two multisets behave like two heaps that *also* support $O(\log k)$ removal of a specific value, keeping the median at the boundary between the halves.

Algorithm

1. Insert into *low* or *high* by comparing with the current median; rebalance so $|low| - |high| \in \{0, 1\}$.
2. On slide, erase the departing value from its half and rebalance.

3. Output `*low.rbegin()` per window.

Dry run

Dry Run. $a = [2, 4, 1, 3]$, $k = 3$.

- $[2, 4, 1]$ sorted 1, 2, 4: low = {1, 2}, high = {4}; median 2.
- Slide to $[4, 1, 3]$ sorted 1, 3, 4: erase 2, insert 3; low = {1, 3}, high = {4}; median 3.
- Output 2, 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    multiset<int> low, high;                                // low: smaller half
11    auto rebalance = [&]() {
12        while (low.size() > high.size() + 1) {
13            high.insert(*low.rbegin());
14            low.erase(prev(low.end()));
15        }
16        while (low.size() < high.size()) {
17            low.insert(*high.begin());
18            high.erase(high.begin());
19        }
20    };
21    auto insert = [&](int x) {
22        if (low.empty() || x <= *low.rbegin()) low.insert(x);
23        else high.insert(x);
24        rebalance();
25    };
26    auto erase = [&](int x) {
27        auto it = low.find(x);
28        if (it != low.end()) low.erase(it);
29        else high.erase(high.find(x));
30        rebalance();
31    };
32
33    for (int i = 0; i < k; i++) insert(a[i]);
34    cout << *low.rbegin() << " ";
35    for (int i = k; i < n; i++) {
36        insert(a[i]); erase(a[i - k]);
37        cout << *low.rbegin() << " ";
38    }
39    cout << "\n";
40    return 0;
41 }

```

Complexity Analysis

Time Complexity. $O(n \log k)$ – each insert/erase/rebalance is $O(\log k)$. **Space Complexity.** $O(k)$.

Key Takeaway

The two-balanced-multisets structure keeps the median at the split point under both insertion and deletion – the erasable-heap the sliding window needs. It directly powers the next problem, sliding window cost, by adding running sums to each half.

1.10 Sliding Window Cost

Problem Statement. Given an array of n integers and a window size k , for every window output the minimum total cost of making all its elements equal, where the cost of changing a value by d is $|d|$. The optimal target is the window's median.

Pattern Recognition

Minimising $\sum |a_i - x|$ is achieved at $x = \text{median}$, so this is the median structure of the previous section *plus running sums*. With the two halves' sizes and sums maintained, the cost is a closed-form expression evaluated in $O(1)$ per window.

Cost from the two halves

Reuse the low/high multisets that keep the median at the boundary, and also maintain *lowSum* and *highSum*. With median x , the elements in *low* are all $\leq x$ and cost $x \cdot |\text{low}| - \text{lowSum}$ to raise; those in *high* are $\geq x$ and cost $\text{highSum} - x \cdot |\text{high}|$ to lower. The total is

$$\text{cost} = (x |\text{low}| - \text{lowSum}) + (\text{highSum} - x |\text{high}|).$$

Every insert/erase updates the corresponding sum alongside the multiset.

The Median Minimises Absolute Deviation

For $\sum |a_i - x|$, moving x toward the median always reduces the total until x reaches it – a standard exchange argument. So the only statistic needed is the median plus the sums of the two halves, all of which the balanced multisets already track.

Algorithm

1. Maintain low/high multisets (median at the split) and their sums.
2. On insert/erase, update sizes and sums.
3. Per window, compute the closed-form cost with $x = *low.rbegin()$.

Dry run

Dry Run. Window $[1, 4, 3]$, median 3.

- $\text{low} = \{1, 3\}$ (sum 4), $\text{high} = \{4\}$ (sum 4), $x = 3$.
- $\text{cost} = (3 \cdot 2 - 4) + (4 - 3 \cdot 1) = 2 + 1 = 3$ – raise 1 $\rightarrow 3$ (cost 2), lower 4 $\rightarrow 3$ (cost 1).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<long long> a(n);
8     for (auto& x : a) cin >> x;
9
10    multiset<long long> low, high;
11    long long lowSum = 0, highSum = 0;
12    auto rebalance = [&]() {
13        while (low.size() > high.size() + 1) {
14            long long v = *low.rbegin(); low.erase(prev(low.end()));
15            lowSum -= v; high.insert(v); highSum += v;
16        }
17        while (low.size() < high.size()) {
18            long long v = *high.begin(); high.erase(high.begin());
19            highSum -= v; low.insert(v); lowSum += v;
20        }
21    };
22    auto insert = [&](long long x) {
23        if (low.empty() || x <= *low.rbegin()) { low.insert(x); lowSum
24            += x; }
25        else { high.insert(x); highSum += x; }
26        rebalance();
27    };
28    auto erase = [&](long long x) {
29        auto it = low.find(x);
30        if (it != low.end()) { low.erase(it); lowSum -= x; }
31        else { high.erase(high.find(x)); highSum -= x; }
32        rebalance();
33    };
34    auto cost = [&]() {
35        long long x = *low.rbegin();
36        return (x * (long long)low.size() - lowSum) +
37            (highSum - x * (long long)high.size());
38    };
39    for (int i = 0; i < k; i++) insert(a[i]);
40    cout << cost() << " ";
41    for (int i = k; i < n; i++) {
42        insert(a[i]); erase(a[i - k]);
43        cout << cost() << " ";
44    }
45    cout << "\n";
46    return 0;
47 }

```

Complexity Analysis

Time Complexity. $O(n \log k)$ – median maintenance dominates. **Space Complexity.** $O(k)$.

Key Takeaway

Minimising absolute deviation is a median problem; carrying the two halves' sums alongside the median turns the cost into a one-line formula. Augmenting an order-statistic structure with aggregate sums is a broadly useful move.

1.11 Sliding Window Inversions

Problem Statement. Given an array of n integers and a window size k , output the number of inversions in every contiguous window – pairs of positions $i < j$ within the window with $a_i > a_j$.

Pattern Recognition

Recomputing inversions per window is $O(nk \log k)$. Instead maintain a running inversion count and a **Fenwick tree** over value ranks: adding the new right element and removing the old left element each change the count by a range query answerable in $O(\log n)$.

Incremental inversion count

Keep a Fenwick tree of the value counts currently in the window and a running total inv . When a new element x enters on the *right*, it forms an inversion with every current element strictly greater than x (all of which sit to its left), so $inv += (\text{window size}) - \#\{\leq x\}$; then insert x . When the leftmost element y leaves, it was to the left of everything else, so it formed an inversion with every current element strictly less than y : $inv -= \#\{< y\}$; then remove y . Coordinate-compress values so ranks fit the Fenwick tree.

Add on the Right, Remove on the Left, Query the Rank

The window's geometry makes the updates clean: a right-entering element pairs with larger existing elements; a left-leaving element pairs with smaller ones. Both counts are Fenwick prefix queries, so each slide is $O(\log n)$.

Algorithm

1. Compress values to ranks $1 \dots m$. Build the first window, adding elements and accumulating inversions.
2. Per slide: remove the leftmost ($inv -= \text{count of smaller present}$), then add the new ($inv += \text{count of larger present}$).
3. Output inv per window.

Dry run

Dry Run. $a = [2, 1, 3]$, $k = 2$.

- Window $[2, 1]$: $2 > 1$ is one inversion $\Rightarrow 1$.
- Slide to $[1, 3]$: remove 2 (it had 1 smaller element $\rightarrow inv = 0$), add 3 (nothing greater present $\rightarrow$ stays 0).
- Output 1, 0.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct BIT {
5     int n; vector<int> t;
6     BIT(int n) : n(n), t(n + 1, 0) {}
7     void update(int i, int v) { for (; i <= n; i += i & -i) t[i] += v;
8         }
9     int query(int i) { int s = 0; for (; i > 0; i -= i & -i) s += t[i];
10         return s; }
11 };
12
13 int main() {
14     int n, k;
15     cin >> n >> k;
16     vector<int> a(n);
17     for (int& x : a) cin >> x;
18
19     vector<int> vals(a);
20     sort(vals.begin(), vals.end());
21     vals.erase(unique(vals.begin(), vals.end()), vals.end());
22     auto rankOf = [&](int x) {
23         return int(lower_bound(vals.begin(), vals.end(), x) - vals.
24             begin()) + 1;
25     };
26     int m = vals.size();
27     BIT bit(m);
28
29     long long inv = 0;
30     int inWin = 0;
31     for (int i = 0; i < k; i++) { // build first window
32         int r = rankOf(a[i]);
33         inv += inWin - bit.query(r); // existing elements > a
34         bit.update(r, +1); inWin++;
35     }
36     cout << inv << " ";
37     for (int i = k; i < n; i++) {
38         int ry = rankOf(a[i - k]); // remove leftmost
39         inv -= bit.query(ry - 1); // present elements < y
40         bit.update(ry, -1); inWin--;
41
42         int rx = rankOf(a[i]); // add new right
43         inv += inWin - bit.query(rx); // present elements > x
44         bit.update(rx, +1); inWin++;
45         cout << inv << " ";
46     }
47     cout << "\n";
48     return 0;
49 }

```

Complexity Analysis

Time Complexity. $O(n \log n)$ – each slide does a constant number of Fenwick operations.

Space Complexity. $O(n)$ for the Fenwick tree and compression.

Key Takeaway

Sliding-window inversions become linear-log by maintaining a running count plus a Fenwick tree of present values: the right-entering element pairs with larger elements, the left-leaving with smaller. Recognising which side an element joins/leaves fixes which prefix query to run.

1.12 Sliding Window Advertisement (Maximum)

Problem Statement. Given an array of n integers and a window size k , output the maximum of every contiguous window. (This “advertisement” variant is the maximum counterpart of the sliding window minimum.)

Pattern Recognition

Maximum, like minimum, is not invertible, so the same **monotonic deque** applies – kept in *decreasing* order this time, so the front is always the current window’s maximum. Each index is pushed and popped once, giving $O(n)$.

A decreasing deque

Maintain a deque of indices whose values are strictly decreasing from front to back. When a new element arrives, pop from the back every index whose value is $\leq$ the new value – those are dominated and can never be the maximum again while the newer, larger element is present. Push the new index, drop the front if it has left the window, and read the front as the window maximum.

Mirror of the Minimum Deque

The only change from sliding window minimum is the comparison direction: pop while the back is $\leq$ the incoming value (instead of $\geq$), keeping the deque decreasing so its front is the maximum. One structure, two mirror uses.

Algorithm

1. For each index i : pop back while its value $\leq a[i]$; push i .
2. Pop front while it is $\leq i - k$ (out of window).
3. Once $i \geq k - 1$, output $a[\text{front}]$.

Dry run

Dry Run. $a = [3, 1, 4, 2]$, $k = 2$.

- $[3, 1]$: deque front 3; max 3.
- $[1, 4]$: 4 pops 1 (and 3 leaves window); max 4.
- $[4, 2]$: front 4; max 4. Output 3, 4, 4.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    deque<int> dq;                                // indices, values
11    decreasing
12    for (int i = 0; i < n; i++) {
13        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
14        dq.push_back(i);
15        if (dq.front() <= i - k) dq.pop_front();
16        if (i >= k - 1) cout << a[dq.front()] << " ";
17    }
18    cout << "\n";
19    return 0;
20 }

```

Complexity Analysis

Time Complexity. $O(n)$ – each index enters and leaves the deque once. **Space Complexity.** $O(k)$.

Key Takeaway

Sliding window maximum is the minimum deque mirrored: pop while the back is no larger than the incoming value. Master the monotonic deque once and both extremes – and many “nearest greater/smaller” problems – follow from it.

Chapter 2

Two Pointers and Subarrays

The sliding window’s cousin, the two-pointer method, and its algebraic sibling, the prefix sum, solve the “count or find a subarray with a property” family. When the property grows monotonically as the window widens, two moving pointers suffice; when it does not, a prefix sum plus a hash map turns the search into counting equal keys. These problems from the Sorting and Searching section round out the window toolkit.

2.1 Two Pointers vs Prefix Sums

Both techniques avoid examining all $O(n^2)$ subarrays, but they apply in different regimes.

Which Technique for Which Subarray Problem?

- **Two pointers** – when a window’s property is *monotonic*: widening the right end only ever increases the sum (positive values) or the distinct count, so the left end can chase it without ever moving backward. Used for Subarray Sums I, Subarray Distinct Values, Playlist.
- **Prefix sums + hash map** – when values can be negative or the target is exact equality, monotonicity is lost; instead count how many earlier prefixes give the needed difference or residue. Used for Subarray Sums II, Subarray Divisibility.
- **Running best (Kadane)** – when maximising a subarray sum, a single scan tracking “best ending here” suffices: Maximum Subarray Sum.

Prefix Sums Turn Subarray Questions into Pair Questions

The sum of $a[l..r]$ is $pre[r] - pre[l - 1]$. So “a subarray with sum x ” becomes “two prefixes differing by x ,” countable in one pass with a hash map of prefix values. This transform is the backbone of the exact-sum and divisibility problems.

2.2 Subarray Sums I

Problem Statement. Given an array of n *positive* integers and a target x , count the number of contiguous subarrays whose sum equals x .

Pattern Recognition

With all values positive, the window sum is **monotonic**: extending the right end only increases it, and advancing the left end only decreases it. That monotonicity is exactly what lets a **two-pointer** sliding window count matches in one linear pass.

Why two pointers work here

Keep a window $[l, r]$ and its sum. Advance r , adding $a[r]$. Whenever the sum exceeds x , advance l , subtracting $a[l]$ – safe because positivity guarantees shrinking only lowers the sum. Whenever the sum equals x , record one subarray. Because the sum never decreases when r moves right and never increases when l moves right, each pointer only moves forward, giving $O(n)$.

Positivity Makes the Window Monotone

Two pointers require that growing the window changes the tracked quantity in one direction only. Positive values guarantee that for sums – which is why Subarray Sums II (negatives allowed) must abandon two pointers for prefix sums.

Algorithm

1. $l = 0$, $sum = 0$, $count = 0$.
2. For $r = 0 \dots n - 1$: add $a[r]$; while $sum > x$ advance l subtracting $a[l]$; if $sum = x$ increment $count$.
3. Output $count$.

Dry run

Dry Run. $a = [2, 4, 1, 2, 7]$, $x = 7$.

- $[2, 4, 1]$ sums to 7 $\Rightarrow$ count 1.
- Window shrinks/extends; $[4, 1, 2] = 7 \Rightarrow$ count 2; later $[7] = 7 \Rightarrow$ count 3.
- Answer 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n; long long x;
6     cin >> n >> x;
7     vector<long long> a(n);
8     for (auto& v : a) cin >> v;
9
10    long long sum = 0, count = 0;
11    int l = 0;

```

```

12     for (int r = 0; r < n; r++) {
13         sum += a[r];
14         while (sum > x) sum -= a[l++];           // positivity: safe to
15             shrink
16         if (sum == x) count++;
17     }
18     cout << count << "\n";
19     return 0;

```

Complexity Analysis

Time Complexity. $O(n)$ – both pointers only move forward. **Space Complexity.** $O(1)$.

Key Takeaway

Two pointers count target-sum subarrays in linear time precisely because positive values keep the window sum monotone. The moment negatives enter, monotonicity breaks and the prefix-sum-plus-hash-map method takes over.

2.3 Subarray Sums II

Problem Statement. Given an array of n integers (which may be negative) and a target x , count the number of contiguous subarrays whose sum equals x .

Pattern Recognition

With negatives, the window sum is no longer monotone, so two pointers fail. Instead use **prefix sums plus a hash map**: a subarray $[l + 1, r]$ has sum x iff $pre[r] - pre[l] = x$, so count, for each prefix, how many earlier prefixes equal $pre[r] - x$.

Counting equal prefix differences

Let $pre[i]$ be the sum of the first i elements. A subarray ending at position r with sum x corresponds to an earlier prefix $pre[l] = pre[r] - x$. Sweep left to right maintaining a hash map from prefix-sum value to how many times it has occurred; at each step add the map's count of $pre[r] - x$ to the answer, then record $pre[r]$. Initialise the map with $\{0 : 1\}$ so subarrays starting at index 0 are counted.

Turn a Subarray Sum into a Prefix Difference

Any subarray sum is a difference of two prefix sums, so “sum equals x ” becomes “two prefixes differ by x .” A hash map of previously seen prefix values counts those pairs in one pass, negatives and all.

Algorithm

1. Map $\{0 : 1\}$, $pre = 0$, $count = 0$.
2. For each element: $pre += a[i]$; add $map[pre - x]$ to $count$; increment $map[pre]$.
3. Output $count$.

Dry run

Dry Run. $a = [2, -1, 3, 1]$, $x = 4$.

- Prefixes: 0, 2, 1, 4, 5. Looking for earlier prefix $pre - 4$.
- At $pre = 4$: $pre - 4 = 0$ seen once $\Rightarrow$ count 1 ($[2, -1, 3]$). At $pre = 5$: $5 - 4 = 1$ seen once $\Rightarrow$ count 2 ($[3, 1]$).
- Answer 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n; long long x;
6     cin >> n >> x;
7     unordered_map<long long, long long> seen;
8     seen[0] = 1; // empty prefix
9
10    long long pre = 0, count = 0;
11    for (int i = 0; i < n; i++) {
12        long long a; cin >> a;
13        pre += a;
14        auto it = seen.find(pre - x); // earlier prefix that
15        // closes a sum-x subarray
16        if (it != seen.end()) count += it->second;
17        seen[pre]++;
18    }
19    cout << count << "\n";
20    return 0;
}

```

Complexity Analysis

Time Complexity. $O(n)$ amortised with a hash map. **Space Complexity.** $O(n)$ for the map.

Key Takeaway

Once negatives break monotonicity, prefix sums plus a hash map count exact-sum subarrays in one pass – the go-to when two pointers cannot apply. This is the same machinery behind “subarray with sum divisible by n ” (next section), swapping the key from a value to a residue.

2.4 Subarray Divisibility

Problem Statement. Given an array of n integers, count the number of contiguous subarrays whose sum is divisible by n .

Pattern Recognition

“Sum divisible by n ” is prefix sums modulo n : a subarray sum is a multiple of n iff its two bounding prefix sums have the **same residue** mod n . So count pairs of equal residues – a

hash map (or array) of residue frequencies does it in one pass.

Equal residues bound divisible subarrays

$sum(l+1, r) = pre[r] - pre[l]$ is divisible by n exactly when $pre[r] \equiv pre[l] \pmod{n}$. So for each residue class, any two prefixes in it form a valid subarray. Sweeping the prefixes and tallying how many share each residue, the number of divisible subarrays is $\sum_r \binom{c_r}{2}$ where c_r counts prefixes with residue r (including the empty prefix, residue 0). Handle negative sums by normalising the residue into $[0, n)$.

Residue Collisions Are Divisible Subarrays

Reducing prefix sums mod n converts “divisible difference” into “equal residue.” Counting how many prefixes fall in each residue class, then choosing 2 from each class, counts every divisible subarray without examining pairs directly.

Algorithm

1. Frequency array $cnt[0 \dots n-1]$ with $cnt[0] = 1$ (empty prefix).
2. Running prefix mod n (normalised non-negative); for each element add $cnt[residue]$ to the answer, then increment that residue’s count.
3. Output the accumulated count.

Dry run

Dry Run. $a = [1, 2, 3, 4, 5]$, $n = 5$.

- Prefixes mod 5: 0, 1, 3, 1, 0, 0.
- Residue 0 occurs at prefixes (empty), 4, 5 – three of them; residue 1 twice.
- Answer $\binom{3}{2} + \binom{2}{2} = 3 + \dots$ counted incrementally to the correct total of divisible subarrays.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<long long> cnt(n, 0);
8     cnt[0] = 1;                                     // empty prefix has
6     residue 0
9
10    long long pre = 0, ans = 0;
11    for (int i = 0; i < n; i++) {
12        long long a; cin >> a;
13        pre = ((pre + a) % n + n) % n;               // normalise residue to
14        [0, n)
15        ans += cnt[pre];                             // earlier prefixes with
16        same residue
17        cnt[pre]++;

```

```

16     }
17     cout << ans << "\n";
18     return 0;
19 }
```

Complexity Analysis

Time Complexity. $O(n)$. **Space Complexity.** $O(n)$ for the residue counts.

Key Takeaway

Divisibility of a subarray sum is equality of prefix residues; count residue collisions in one pass. This is Subarray Sums II with the map key changed from a raw prefix value to its residue mod n – a small twist on the same prefix-and-count template.

2.5 Subarray Distinct Values

Problem Statement. Given an array of n integers and an integer k , count the number of contiguous subarrays that contain at most k distinct values.

Pattern Recognition

“At most k distinct” is monotone – shrinking a window never increases its distinct count – so a **variable-size two-pointer window** works: for each right end, advance the left end until at most k distinct remain, and every subarray ending here within the window is valid.

Count subarrays ending at each right end

Maintain a window $[l, r]$ with a frequency map and its distinct count. Extend r ; if adding $a[r]$ pushes the distinct count above k , advance l (removing elements) until it drops back to k . Now every subarray that ends at r and starts anywhere in $[l, r]$ has at most k distinct values, contributing $r - l + 1$ subarrays. Summing that over all r gives the total.

Window Width Is the Count of Valid Subarrays

Because the “at most k distinct” property is preserved under shrinking, once the window $[l, r]$ is valid, so is every subarray ending at r with a start $\geq l$. Adding $r - l + 1$ per right end counts them all without enumeration.

Algorithm

1. $l = 0$, empty frequency map, $distinct = 0$, $ans = 0$.
2. For $r = 0 \dots n - 1$: add $a[r]$; while $distinct > k$ remove $a[l]$ and advance l ; add $r - l + 1$ to ans .
3. Output ans .

Dry run

Dry Run. $a = [1, 2, 1, 3]$, $k = 2$.

- $r = 0$: window $[1]$, add 1. $r = 1$: $[1, 2]$, add 2. $r = 2$: $[1, 2, 1]$ (distinct 2), add 3.
- $r = 3$: adding 3 gives distinct 3; shrink to $[2, 1, 3]$? still 3; shrink to $[1, 3]$, distinct 2; add 2.

- Total = $1 + 2 + 3 + 2 = 8$ subarrays with at most 2 distinct.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, k;
6     cin >> n >> k;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    unordered_map<int, int> freq;
11    long long ans = 0;
12    int l = 0, distinct = 0;
13    for (int r = 0; r < n; r++) {
14        if (freq[a[r]]++ == 0) distinct++;
15        while (distinct > k) {
16            if (--freq[a[l]] == 0) distinct--;
17            l++;
18        }
19        ans += r - l + 1;           // subarrays ending at r
20    }
21    cout << ans << "\n";
22    return 0;
23 }

```

Complexity Analysis

Time Complexity. $O(n)$ amortised – each pointer moves forward at most n times. **Space Complexity.** $O(k)$ distinct keys.

Key Takeaway

“Count subarrays with at most k (property)” is a variable window where each right end contributes its window width. To count *exactly* k distinct, take $\text{atMost}(k) - \text{atMost}(k - 1)$ – a standard follow-up on the same routine.

2.6 Playlist

Problem Statement. Given a playlist of n songs (each an integer id), find the length of the longest contiguous stretch of songs with *no repeats* (all values distinct).

Pattern Recognition

“Longest window with all distinct” is a **variable two-pointer window**: extend the right end, and whenever a duplicate appears, jump the left end past the duplicate’s previous occurrence so the window is distinct again. Track the best width seen.

Jump the left end past duplicates

Maintain a map from each value to its most recent index. Advance r ; if $a[r]$ was seen at some index $\geq l$, move l to one past that index – this discards exactly the earlier copy and everything

before it, restoring distinctness in one jump rather than shrinking one step at a time. After each step the window $[l, r]$ is duplicate-free, so update the best length with $r - l + 1$.

Last-Seen Index Enables an $O(1)$ Left Jump

Instead of shrinking the window element by element, remembering each value's last position lets the left pointer leap directly to the smallest valid start. Both pointers still only move forward, keeping the scan linear.

Algorithm

1. $l = 0$, empty last-seen map, $best = 0$.
2. For $r = 0 \dots n - 1$: if $a[r]$'s last index $\geq l$, set $l = \text{lastIndex} + 1$; update $\text{last}[a[r]] = r$; update $best = \max(best, r - l + 1)$.
3. Output $best$.

Dry run

Dry Run. $a = [1, 2, 1, 3, 2]$.

- $r = 2$ ($a = 1$): previous 1 at index $0 \geq l = 0$, jump $l = 1$; window $[2, 1]$.
- $r = 4$ ($a = 2$): previous 2 at index $1 < l$? l is now 1, so $1 \geq 1$, jump $l = 2$; window $[1, 3, 2]$, length 3.
- Best length = 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    unordered_map<int, int> last;           // value -> last index
11    seen
12    int l = 0, best = 0;
13    for (int r = 0; r < n; r++) {
14        auto it = last.find(a[r]);
15        if (it != last.end() && it->second >= l) l = it->second + 1;
16        // jump past dup
17        last[a[r]] = r;
18        best = max(best, r - l + 1);
19    }
20    cout << best << "\n";
21    return 0;
22 }
```

Complexity Analysis

Time Complexity. $O(n)$ amortised. **Space Complexity.** $O(n)$ for the last-seen map.

Key Takeaway

Longest-distinct-window is a two-pointer scan where a last-seen map lets the left end jump directly past a repeat. The same “remember last position, jump the left bound” idiom solves longest-substring-without-repeat and many uniqueness-constrained window problems.

2.7 Maximum Subarray Sum

Problem Statement. Given an array of n integers (possibly negative), find the maximum sum of any non-empty contiguous subarray.

Pattern Recognition

The maximum-subarray problem is solved by **Kadane’s algorithm**: a single scan maintaining the best subarray sum *ending at the current position*, resetting whenever carrying the previous sum forward would only hurt.

Best sum ending here

Let cur be the maximum sum of a subarray ending exactly at the current index. Extending to the next element, either continue the previous subarray ($cur + a[i]$) or start fresh at $a[i]$ – whichever is larger:

$$cur = \max(a[i], cur + a[i]).$$

The overall answer is the maximum cur over all positions. Starting fresh happens exactly when the running sum has gone negative and would drag down any extension.

Drop a Negative Prefix

A prefix with negative sum can never help a later subarray – discarding it (resetting cur to $a[i]$) is always at least as good. This single greedy reset is what makes one linear pass optimal.

Algorithm

1. $cur = best = a[0]$.
2. For $i = 1 \dots n - 1$: $cur = \max(a[i], cur + a[i])$; $best = \max(best, cur)$.
3. Output $best$.

Dry run

Dry Run. $a = [-1, 3, -2, 5, -4]$.

- cur : $-1 \rightarrow 3 \rightarrow 1 \rightarrow 6 \rightarrow 2$. Resets never beat carrying here except at the start.
- $best = 6$ (the subarray $[3, -2, 5]$).

C++ implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<long long> a(n);
8     for (auto& x : a) cin >> x;
9
10    long long cur = a[0], best = a[0];
11    for (int i = 1; i < n; i++) {
12        cur = max(a[i], cur + a[i]);           // extend or restart
13        best = max(best, cur);
14    }
15    cout << best << "\n";
16    return 0;
17 }
```

Complexity Analysis

Time Complexity. $O(n)$ – a single pass. **Space Complexity.** $O(1)$.

Key Takeaway

Kadane’s algorithm is the definitive maximum-subarray solution: keep the best sum ending here, restart whenever the running total turns negative. Its “discard a harmful prefix” idea recurs in maximum-product subarray and several DP-on-a-line problems.